

DevOps Interview Scenarios

Complete Scenario-Based Answers with Commands & Explanations

Prepared for: Sanket Ajay Chopade

DevOps Engineer | AWS · Kubernetes · Terraform · Jenkins · GitOps

Scenarios Covered	26 real-world scenarios + deep cross-questions
Topics	Kubernetes · EKS · Terraform · Jenkins · Docker · IAM · GitOps · Monitoring
Level	Junior / Fresher DevOps Engineer (Internship-based)
Total Sections	12 topic sections + 1 cross-question section

SECTION 1 — End-to-End Production Deployment Scenarios

Scenario 1: Deployment Failure — Jenkins Succeeds but App Is Unavailable

A developer merges code into main. Jenkins pipeline reports success. Users report the application is unavailable after deployment.

■ How would you start troubleshooting?

- First, check the pod status to see if pods are running, crashing, or pending.

```
kubectl get pods -n <namespace>
```

- Look at recent Kubernetes events to spot any errors or restarts.

```
kubectl get events -n <namespace> --sort-by='.lastTimestamp'
```

- Describe the deployment to see replicas, conditions, and recent events.

```
kubectl describe deployment <deployment-name> -n <namespace>
```

■ Which logs would you check first?

- Application pod logs — this is almost always the first place to look.

```
kubectl logs <pod-name> -n <namespace>
```

- If the pod has already restarted, check logs from the previous (crashed) instance.

```
kubectl logs <pod-name> --previous -n <namespace>
```

- Kubernetes events — they show OOMKilled, image pull errors, readiness failures, etc.

■ How would you determine whether the issue is application-related or infrastructure-related?

- Application-related signs: CrashLoopBackOff, non-zero exit codes, error messages in pod logs.
- Infrastructure-related signs: nodes NotReady, PVC not bound, ALB not created, pod stuck in Pending.
- Use 'kubectl describe pod' — the Events section clearly separates app vs infra errors.

```
kubectl describe pod <pod-name> -n <namespace>
```

■ What Kubernetes commands would you use?

```
kubectl get pods -n <namespace> -o wide
```

```
kubectl describe pod <pod-name> -n <namespace>
```

```
kubectl logs <pod-name> --tail=100 -n <namespace>
```

```
kubectl get events -n <namespace>
```

```
kubectl get svc -n <namespace>
```

```
kubectl describe ingress -n <namespace>
```

■ How would you rollback the deployment?

- Use kubectl rollout undo to revert to the previous deployment revision.

```
kubectl rollout undo deployment/<name> -n <namespace>
```

- To rollback to a specific revision:

```
kubectl rollout undo deployment/<name> --to-revision=<N>
```

- Verify the rollback is successful:

```
kubectl rollout status deployment/<name>
```

■ How would Argo CD help in this situation?

- Argo CD UI shows the exact Git commit that was deployed and its health status.
- The 'diff' view shows what changed between the last good version and the current one.
- You can rollback to any previous deployment with one click directly in the Argo CD UI.
- If someone made manual changes via kubectl, Argo CD marks the app as OutOfSync — alerting you immediately.

■ How would you prevent this in future releases?

- Add readiness probes — Kubernetes will not route traffic until the new pod is truly ready.
- Add liveness probes — Kubernetes will restart the pod if it becomes unhealthy.

- Implement canary or blue-green deployments to gradually shift traffic to the new version.
- Add post-deployment smoke tests in the CI/CD pipeline to catch failures before closing the pipeline.
- Use Argo CD sync waves and hooks for ordered, safe deployments.

Scenario 2: Jenkins Pipeline Failure — Suddenly Starts Failing After Months

■ Which stage failed?

- Open the failed build in Jenkins and read the Console Output.
- Jenkins pipeline view highlights the exact failed stage in red — Checkout, Build, Test, SonarQube, Docker Build, Push, or Deploy.
- Error message in the output tells you the exact reason (e.g., 'Cannot connect to Docker daemon', 'Quality Gate failed').

■ How would you isolate the root cause?

- Read the full error message in the failed stage — don't guess.
- Check if any recent change was made: Jenkinsfile update, plugin update, dependency change, or credentials rotation.
- Compare the current build with the last successful build — what changed?

■ What if SonarQube integration is causing the failure?

- Check if the SonarQube server is accessible — could be down or unreachable.
- Verify the SonarQube token stored in Jenkins Credentials hasn't expired.
- If Quality Gate is failing, it's a code issue (not infra) — the pipeline is working correctly.
- Check SonarQube plugin version in Jenkins — a recent plugin update may have changed behaviour.

■ What if Docker build suddenly fails?

- Check if the base image in the Dockerfile is still accessible (Docker Hub rate limits, deleted image).
- Check disk space on the Jenkins agent — a full disk breaks builds silently.
`df -h`
- Check if the Docker daemon is running on the agent:
`systemctl status docker`
- Check for syntax errors or changed dependencies in the Dockerfile.

■ What if GitHub webhook is not triggering Jenkins?

- Open the GitHub repo → Settings → Webhooks → check delivery logs for errors.
- Verify the Jenkins URL in the webhook is correct and reachable from GitHub (firewall/network issue?).
- Check the shared secret between GitHub and Jenkins matches.
- As a test, manually trigger the Jenkins build to confirm the pipeline itself is fine.

■ How would you troubleshoot Jenkins agents?

- Go to Manage Jenkins → Manage Nodes — check if the agent shows as online.
- Look at the agent's log from the Jenkins UI for connection errors.
- SSH into the agent machine and check disk space, memory, and Docker daemon status.
- If using Kubernetes-based agents, check if the agent pods are being created:
`kubectl get pods -n jenkins`

SECTION 2 — Kubernetes Troubleshooting Scenarios

Scenario 3: Pod in CrashLoopBackOff

■ What does CrashLoopBackOff mean?

- The container starts, crashes (exits with a non-zero code), and Kubernetes automatically restarts it.
- After each crash, Kubernetes waits longer before the next restart (exponential backoff — 10s, 20s, 40s...).
- Common causes: application error on startup, missing config, wrong command, insufficient memory.

■ What commands would you run?

```
kubectl get pods -n <namespace>
kubectl describe pod <pod-name> -n <namespace>
kubectl logs <pod-name> -n <namespace>
kubectl logs <pod-name> --previous -n <namespace>
```

■ How would you inspect logs?

- 'kubectl logs <pod-name>' shows current container logs.
- '--previous' flag shows logs from the last crashed container — crucial for diagnosing startup failures.
- 'kubectl describe pod' shows the Events section — look for exit codes, OOMKilled, failed mounts.

■ What if logs are empty?

- Container may be crashing before it can write any output.
- Check the exit code in 'kubectl describe pod' — exit code 137 = OOMKilled (out of memory).
- Try running the container image locally with the same command to reproduce the crash.
- Check if the container command/entrypoint is correct in the pod spec.

■ What if the application starts successfully locally?

- The most likely culprits are environment variables — your local .env file doesn't exist in Kubernetes.
- Check if required Secrets or ConfigMaps exist in the cluster and are mounted correctly.
- Check if resource limits (CPU/memory) are too low — K8s may be killing the container.
- Network differences — service names, DNS resolution work differently in Kubernetes.

```
kubectl describe pod <pod-name> -n <namespace> # check env, mounts, events
```

■ Could ConfigMaps cause this issue?

- Yes. If a ConfigMap referenced in the pod spec doesn't exist, the pod will not start at all.
- If a ConfigMap has wrong values (wrong DB host, bad port), the app may crash on startup.

```
kubectl get configmap -n <namespace>
kubectl describe configmap <name> -n <namespace>
```

■ Could Secrets cause this issue?

- Yes. Missing Secrets are a very common cause. If a Secret referenced as a volume or env var doesn't exist, the pod never starts.
 - Wrong credentials in a Secret cause the app to crash when it tries to connect to the database.
- ```
kubectl get secrets -n <namespace>
```
- Check if the Secret is mounted correctly in 'kubectl describe pod'.

### Scenario 4: Pod Stuck in Pending State

#### ■ What causes a Pod to remain in Pending state?

- Insufficient CPU or memory on all available nodes.
- PVC (PersistentVolumeClaim) is not yet bound to a PersistentVolume.
- Node selector or affinity rules don't match any available node.
- Nodes have taints and the pod has no matching tolerations.

- Image pull failure — wrong image name, private registry without pull secret.

#### ■ How would you check scheduler events?

- Describe the pod — the Events section shows the scheduler's reason for not placing the pod.  
`kubectl describe pod <pod-name> -n <namespace>`
- Common message: '0/3 nodes are available: insufficient memory' — tells you exactly what's missing.

#### ■ What if nodes have insufficient resources?

- Check current node resource usage:  
`kubectl top nodes`
- Check allocatable vs requested resources on a specific node:  
`kubectl describe node <node-name>`
- Solutions: scale up the EKS node group in AWS Console, or reduce the pod's resource requests.

#### ■ What if PVC is not bound?

- Check PVC status — it should show 'Bound', not 'Pending':  
`kubectl get pvc -n <namespace>`
- Describe the PVC to see why it's not binding:  
`kubectl describe pvc <pvc-name> -n <namespace>`
- In EKS, ensure the EBS CSI Driver is installed — without it, dynamic provisioning fails.
- Check that the StorageClass exists:  
`kubectl get storageclass`

#### ■ What if taints prevent scheduling?

- Check node taints:  
`kubectl describe node <node-name> | grep Taint`
- Add matching tolerations to the pod spec so the scheduler can place the pod on that node.
- Alternatively, remove the taint if it's not needed:  
`kubectl taint node <node-name> <key>-`

### Scenario 5: Service Not Accessible (Pods Running, Service + Ingress Exist)

#### ■ Where do you begin troubleshooting?

- Verify pods are not just Running but also Ready (1/1 or 2/2 under READY column).  
`kubectl get pods -n <namespace>`
- Check if the Service has endpoints — if empty, the Service selector doesn't match pod labels.  
`kubectl get endpoints <service-name> -n <namespace>`
- Describe the Ingress to check rules and events:  
`kubectl describe ingress -n <namespace>`

#### ■ How do you verify Service endpoints?

- An empty ENDPOINTS column means the Service selector and pod labels don't match.
- Compare the Service selector with actual pod labels:  
`kubectl describe svc <service-name> -n <namespace>`  
`kubectl get pods --show-labels -n <namespace>`
- Fix: correct the label in either the pod spec or the Service selector.

#### ■ How do you verify Ingress configuration?

- Describe the Ingress and read the Rules section — check host, paths, and backend service name.
- The backend service name and port must exactly match an existing Service.
- Check the Events section for errors from the Ingress Controller.  
`kubectl describe ingress <name> -n <namespace>`

#### ■ How would you check AWS Load Balancer Controller?

- Check if the controller is running:  
`kubectl get pods -n kube-system | grep aws-load-balancer`

- Read controller logs for errors:

```
kubectl logs -n kube-system -l app.kubernetes.io/name=aws-load-balancer-controller --tail=50
```

- Verify the ALB is created in AWS Console → EC2 → Load Balancers.

#### ■ What if ALB is not created?

- The ALB Controller may not be running or it may be crashing — check its logs.
- The IAM role for the ALB Controller may be missing required permissions.
- Ingress annotations may be incorrect — must have: 'kubernetes.io/ingress.class: alb' (or ingressClassName: alb).
- Verify the IRSA (IAM Role for Service Accounts) is configured correctly for the controller.

#### ■ What if security groups are blocking traffic?

- Check the ALB Security Group in AWS Console → allow inbound HTTP (80) / HTTPS (443) from 0.0.0.0/0.
  - Check the Node Security Group — it must allow inbound traffic from the ALB Security Group.
  - Check the RDS Security Group if the issue is backend connectivity.
  - AWS VPC Flow Logs can show rejected traffic to confirm a security group is the issue.
-

## SECTION 3 — Amazon EKS Scenarios

### Scenario 6: EKS Worker Node Becomes NotReady

#### ■ What happens to running Pods?

- Pods on the affected node are marked as 'Unknown' or 'Terminating'.
- After a default timeout of approximately 5 minutes, the Node Controller evicts those pods.
- Kubernetes then reschedules evicted pods on healthy nodes (if sufficient resources exist).

#### ■ How would Kubernetes react?

- Node controller marks the node as NotReady after it stops sending heartbeats.
- After the pod eviction timeout, pods are forcibly evicted and rescheduled.
- If your Deployment has multiple replicas, other replicas on healthy nodes continue serving traffic during this process.

#### ■ How would you investigate?

- Describe the node to see conditions and events:  
`kubectl describe node <node-name>`
- Check the EC2 instance status in AWS Console — it may show a system or instance status failure.
- SSH into the node (if accessible) and check kubelet status:  
`systemctl status kubelet`
- Check system logs:  
`sudo journalctl -u kubelet -n 100`

#### ■ How do you drain a node?

- Draining safely evicts all pods from the node before you take it offline for maintenance or replacement.  
`kubectl drain <node-name> --ignore-daemonsets --delete-emptydir-data`
- '--ignore-daemonsets' skips DaemonSet pods (which will be re-created automatically).
- '--delete-emptydir-data' allows draining pods that use emptyDir volumes.

#### ■ How would you recover workloads?

- If the node recovers: uncordon it to allow scheduling again.  
`kubectl uncordon <node-name>`
- If permanently failed: terminate the EC2 instance. The EKS Auto Scaling Group will provision a replacement node automatically.
- Verify all pods are rescheduled and running:  
`kubectl get pods -o wide --all-namespaces`

### Scenario 7: EKS Cluster Version Upgrade

#### ■ What preparations would you make?

- Read the official AWS EKS upgrade documentation for the target version — note breaking changes.
- Check add-on compatibility: CoreDNS, kube-proxy, AWS VPC CNI, EBS CSI Driver must all be updated.
- Scan manifests for deprecated API versions using a tool like 'kubent':  
`kubent # Kubernetes No Trouble – detects deprecated APIs`
- Test the upgrade in a non-production cluster first.
- Take a backup of all Kubernetes resources:  
`kubectl get all --all-namespaces -o yaml > cluster-backup.yaml`

#### ■ How would you minimize downtime?

- The EKS control plane upgrade is done by AWS — it has zero downtime.
- For worker nodes, use managed node groups with rolling updates — old nodes are drained one at a time.
- Set PodDisruptionBudgets (PDB) for critical workloads to ensure minimum availability during node drain.

- Run at least 2 replicas of each critical service so rolling drain doesn't cause downtime.

#### ■ How would you validate workloads afterward?

- Check all pods are running and healthy across all namespaces:  
`kubectl get pods --all-namespaces`
- Verify all add-ons (CoreDNS, kube-proxy, VPC CNI) are updated and running.
- Run smoke tests on critical application endpoints.
- Monitor Grafana/Datadog dashboards for any error rate spikes or performance degradation.

#### ■ What compatibility checks are required?

- Kubernetes version skew policy: kubelet on nodes must be within 2 minor versions of the control plane.
  - Check Helm chart compatibility — some charts have minimum/maximum Kubernetes version requirements.
  - Verify deprecated API versions are removed from all manifests before upgrading.
  - Check container runtime version (containerd) compatibility with the new Kubernetes version.
-



## SECTION 4 — Terraform Scenarios

### Scenario 8: Terraform State Corruption

#### ■ What is Terraform state?

- The state file (terraform.tfstate) is Terraform's record of what infrastructure it has created.
- It maps each resource in your .tf files to a real resource in AWS (or any provider).
- Without state, Terraform doesn't know what already exists and will try to create everything again.

#### ■ What if state gets corrupted?

- 'terraform plan' will fail, show incorrect diffs, or try to recreate already-existing resources.
- Terraform may show resources as 'to be created' that actually exist, or try to delete live resources.
- Any 'terraform apply' run on corrupted state is dangerous — stop immediately.

#### ■ How would you recover?

- Step 1: Restore the state file from the previous S3 version (S3 versioning must be enabled — always use it).
- Step 2: Use 'terraform state' commands to fix individual resources without touching others.  

```
terraform state list # list resources in state
terraform state rm <resource> # remove a bad entry
terraform import <resource> <id> # re-add existing resource to state
```
- Step 3: 'terraform refresh' to sync the state with the actual infrastructure.

#### ■ Why use remote state?

- Team collaboration: all engineers share the same state file — no out-of-sync local copies.
- Safety: state is not lost if a developer's machine crashes or they leave the team.
- Versioning: S3 versioning lets you restore any previous state (essential for recovery).
- Locking: DynamoDB prevents two people from modifying infrastructure at the same time.

#### ■ How does state locking work?

- When 'terraform apply' starts, it writes a lock entry to a DynamoDB table.
- If another engineer runs 'terraform apply' simultaneously, they get an error: 'Error acquiring the state lock'.
- The lock is automatically released when the apply completes (or fails).
- If a process crashes mid-apply, you may need to manually force-unlock:  

```
terraform force-unlock <lock-id>
```

#### ■ What if two engineers run terraform apply simultaneously?

- With remote state + DynamoDB locking: the second apply is blocked until the first finishes. Safe.
- Without DynamoDB locking: both writes can interleave, corrupting the state file. Dangerous.
- Always configure the S3 backend with a DynamoDB table for any team environment.

### Scenario 9: Developer Accidentally Deletes RDS Instance Manually

#### ■ How would Terraform react?

- Terraform's state still contains the RDS resource — Terraform doesn't know it was deleted manually.
- Next 'terraform plan' will show: '1 to add' — Terraform plans to recreate the missing RDS instance.
- If you run 'terraform apply', Terraform will provision a new (empty) RDS instance.

#### ■ How would you recover?

- Option 1 (data exists): Restore the RDS instance from an AWS automated backup or manual snapshot in the AWS Console.
- Option 2 (no data): Run 'terraform apply' to recreate the RDS instance, then restore from any data export if available.

- After restoration, verify the application is pointing to the correct RDS endpoint.

#### ■ What if backups are unavailable?

- Data is permanently lost — this is a data disaster scenario.
- Recreate the RDS instance using Terraform, then restore any data from application-level exports or logs.
- Lesson: always enable AWS automated backups with a retention period of at least 7 days on production RDS.

#### ■ How do you prevent manual changes?

- Use IAM policies to restrict direct deletion of production resources — require Terraform for all changes.
- Add 'lifecycle { prevent\_destroy = true }' in Terraform for critical resources like RDS:  

```
lifecycle { prevent_destroy = true }
```
- Use AWS Config rules to detect and alert on manual changes (resource drift).
- Implement SCP (Service Control Policy) in AWS Organizations to block destructive actions in production accounts.

### Scenario 10: Terraform Apply Stops Midway

#### ■ What would you do?

- Don't panic. Terraform is designed to be idempotent — you can safely re-run it.
- Run 'terraform plan' first to see the current state vs desired state — it shows what was created and what's missing.  

```
terraform plan
```
- Run 'terraform apply' again — Terraform will only create the resources that are still missing. It won't touch already-created ones.

#### ■ How do you verify infrastructure consistency?

- 'terraform plan' shows the diff between your config and real infrastructure — zero changes means consistent.
- 'terraform refresh' syncs the state file with the actual infrastructure (reads current AWS resource state).
- Manually verify critical resources in AWS Console as a final check.

#### ■ Would you edit state manually?

- Only as a last resort, and only using 'terraform state' commands — never edit the raw JSON file.  

```
terraform state list # see all resources in state
terraform state show <resource> # inspect a specific resource
terraform state rm <resource> # remove orphaned resource
terraform import <resource> <aws-id> # add existing resource to state
```

#### ■ What are the risks of manual state editing?

- Invalid JSON in the state file breaks all Terraform operations entirely.
- Incorrect resource IDs can lead to Terraform trying to delete or recreate existing live resources.
- Always make a backup of the state file before any manual operation:  

```
aws s3 cp s3://<bucket>/terraform.tfstate terraform.tfstate.bak
```

## SECTION 5 — AWS IAM Scenarios

### Scenario 11: Application Receives AccessDenied Errors

#### ■ Where would you investigate?

- AWS CloudTrail — it records every API call. Filter for 'AccessDenied' errors to find exactly which action failed.
- Application logs — the error message often includes the exact IAM action that was denied.
- AWS IAM Policy Simulator — test what a specific role can or cannot do.

#### ■ How would you identify the IAM role being used?

- For a pod on EKS: check the pod's service account annotation for the IRSA role:  
`kubectl describe pod <pod-name> -n <namespace>`
- For an EC2 instance: check the instance profile in AWS Console → EC2 → Instance → IAM role.
- Application logs often print the ARN of the role or user that received the access denial.

#### ■ How would you verify trust relationships?

- Go to IAM Console → Roles → select the role → Trust Relationships tab.
- Verify the correct service (EKS, EC2, Lambda) or account is listed as a trusted principal.
- For IRSA (IAM Roles for Service Accounts): verify the OIDC provider is registered for the EKS cluster in IAM.

#### ■ What logs would you check?

- AWS CloudTrail — filter by IAM user/role and error code 'AccessDenied':  
`aws cloudtrail lookup-events --lookup-attributes AttributeKey=ErrorCode,AttributeValue=AccessDenied`
- CloudWatch application logs for the exact denied action (e.g., 's3:GetObject' on bucket 'my-bucket').

#### ■ How would you fix the issue?

- Identify the exact missing permission from the CloudTrail error (e.g., 's3:GetObject').
- Add the required permission to the IAM policy attached to the role.
- If a trust relationship issue: update the trust policy to allow the correct principal to assume the role.
- Use the IAM Policy Simulator to verify the fix before applying to production.

### Scenario 12: IAM User Accidentally Receives AdministratorAccess

#### ■ What risks exist?

- The user now has full access to all AWS services — they can create/delete any resource.
- They could create backdoor IAM users or roles for persistent access.
- They could exfiltrate data from S3 buckets or databases.
- They could launch expensive EC2 instances (crypto mining), causing massive cost.
- This is a compliance violation (PCI-DSS, SOC2, ISO 27001 all require least-privilege).

#### ■ What immediate actions would you take?

- Remove AdministratorAccess policy from the IAM user immediately — this is the top priority.
- Rotate and revoke any access keys belonging to that user.
- Disable the IAM user account if keys may have been seen or shared:  
`aws iam update-login-profile --user-name <user> --password-reset-required`
- Notify your security team and management.

#### ■ How would you audit recent activity?

- AWS CloudTrail → filter by the IAM username for the last 24-48 hours.
- Look for: new IAM users or roles created, S3 bucket access, EC2 instances launched, policy changes.

- Check the AWS Billing Dashboard for unexpected charges from the time of the incident.
- AWS GuardDuty (if enabled) will have alerts for suspicious API calls.

■ **How would you prevent recurrence?**

- Enforce least privilege — only grant permissions actually needed for the user's role.
  - Enable MFA for all IAM users, especially those with any elevated permissions.
  - Use AWS Config rule 'iam-no-inline-policy-check' and alert on AdministratorAccess assignments.
  - Implement permission boundaries to limit the maximum permissions any user can have.
  - Conduct periodic IAM access reviews to remove unused permissions.
-

## SECTION 6 — Docker Scenarios

### Scenario 13: Container Works Locally But Fails in Kubernetes

#### ■ What would you investigate?

- Environment variables: your local `.env` file doesn't exist in Kubernetes — check if all required env vars are set.
- Secrets and ConfigMaps: the app may need database credentials or config that isn't mounted in the pod.
- Resource limits: Kubernetes may be OOMKilling the container due to memory limits.
- Image tag: verify the correct image version is deployed in the pod spec.

#### ■ Could environment variables be the issue?

- Very commonly yes — this is one of the most frequent causes of 'works locally, fails in K8s'.
- Check all environment variables injected into the pod:  
`kubectl describe pod <pod-name> -n <namespace> # look at Environment section`
- Shell into the running container and print all env vars:  
`kubectl exec -it <pod-name> -- env`
- Compare against your local `.env` file — anything missing needs to be added via ConfigMap or Secret.

#### ■ Could Secrets be missing?

- Yes. If the app needs a `DB_PASSWORD`, `API_KEY`, or TLS certificate via a Secret that doesn't exist — the pod won't start or will crash.
- Check if the required Secrets exist:  
`kubectl get secrets -n <namespace>`
- 'kubectl describe pod' shows if a Secret volume failed to mount in the Events section.

#### ■ Could networking be the issue?

- Yes. Kubernetes uses internal DNS — you can't use 'localhost' to refer to other services.
- Correct format: `<service-name>.<namespace>.svc.cluster.local` (e.g., `postgres.default.svc.cluster.local`).
- Network Policies may be blocking traffic between pods — check:  
`kubectl get networkpolicy -n <namespace>`
- External URLs (like a payment gateway) may be blocked by VPC security groups or NACLs.

#### ■ How would you debug?

- Shell into the container (if it's running):  
`kubectl exec -it <pod-name> -- /bin/sh`
- Test internal connectivity from inside the pod:  
`curl http://<service-name>:<port>/health`
- If the pod keeps crashing, deploy a debug container with the same image but override the command to sleep:  
`kubectl run debug --image=<your-image> --command -- sleep 3600`
- Then exec into it and test manually.

### Scenario 14: Docker Image Size Grew from 300 MB to 3 GB

#### ■ How would you investigate?

- Use 'docker history' to see the size of each layer and identify what's bloating the image:  
`docker history <image-name>:<tag>`
- Look for large RUN commands that install dev tools, compilers, or SDKs that aren't needed at runtime.
- Check if `node_modules`, test files, or documentation are being copied into the image.
- Use 'docker image inspect' for full metadata:  
`docker image inspect <image-name>`

### ■ How would you optimize?

- Use multi-stage builds — compile in a build stage, copy only the final artifact to a minimal runtime image.
- Use a minimal base image — Alpine Linux (~5 MB) instead of Ubuntu (~80 MB), or Distroless.
- Add a .dockerignore file to exclude node\_modules, .git, test directories, and logs.
- Combine RUN commands using '&&' to reduce the number of layers (each RUN = one layer):  

```
RUN apt-get update && apt-get install -y curl && apt-get clean && rm -rf /var/lib/apt/lists/*
```
- Run 'apt-get clean' after installing packages to remove cached package files.

### ■ Why use multi-stage builds?

- Build stage contains all tools needed to compile: compilers, SDKs, test frameworks, build tools.
- Final (runtime) stage only contains the compiled binary or artifact — no build tools.
- Example: a Go application — build stage produces a single binary, final stage has just that binary on Distroless.
- Result: a 1 GB build image becomes a 20-30 MB final image — much faster to pull and deploy.

### ■ Why use Distroless images?

- Distroless images contain only the application runtime (e.g., JRE for Java, glibc for Go) — no shell, no package manager.
- Much smaller attack surface — fewer packages means fewer CVEs (security vulnerabilities).
- Trivy security scans show significantly fewer vulnerabilities on Distroless vs Ubuntu/Debian images.

*Note: Your GitOps project on the resume uses Distroless base images — you can cite this directly in interviews.*

---

## SECTION 7 — Monitoring and Observability Scenarios

### Scenario 15: Prometheus Fires High CPU Alert

#### ■ What is your first action?

- Acknowledge the alert to stop repeat notifications.
- Identify which pod or node is consuming high CPU:  

```
kubectl top pods -n <namespace>
```

```
kubectl top nodes
```
- Check if it's a sudden spike (traffic surge, code bug) or a gradual increase (memory leak, slow query).

#### ■ Which Grafana dashboards would you check?

- Node Exporter Full dashboard — shows CPU breakdown per node (user, system, iowait, steal).
- Kubernetes Pod Resources dashboard — shows per-pod CPU usage and limits.
- Application-specific dashboard (if built) — correlate CPU with request rate or queue depth.
- Check if CPU throttling is happening — throttling happens when a pod hits its CPU limit even if overall CPU is normal.

#### ■ How do you determine whether scaling is required?

- If CPU is consistently above 80% of the limit for more than a few minutes → scaling is likely needed.
- Check if HPA has already triggered:  

```
kubectl get hpa -n <namespace>
```
- If it's a code bug (infinite loop, inefficient query) — scaling will not help; you need to fix the code.
- If it's a traffic spike — horizontal scaling (more pods) or vertical scaling (higher limits) is the solution.

#### ■ How does HPA help?

- HPA (Horizontal Pod Autoscaler) automatically increases the number of pod replicas when CPU exceeds a threshold.
- It polls the Kubernetes Metrics Server every 15 seconds and adjusts the replica count.
- Example: set HPA to maintain CPU at 70% utilization, scaling between 2 and 10 replicas:  

```
kubectl autoscale deployment <name> --cpu-percent=70 --min=2 --max=10
```
- HPA also scales down when CPU drops, saving cost.

### Scenario 16: Users Report Application Slowness / High Latency

#### ■ Is CPU high?

- Check pod CPU usage vs limits: `kubectl top pods` — if a pod is at its CPU limit, it gets throttled, causing slowness.
- Even if CPU% looks low, CPU throttling can cause latency — check the 'container\_cpu\_cfs\_throttled\_periods\_total' metric in Prometheus.

#### ■ Is memory high?

- High memory usage causes the OS to swap to disk — this is extremely slow.
- OOMKilled pods (exit code 137) cause restarts and temporary unavailability.  

```
kubectl describe pod <pod-name> | grep OOMKilled
```

#### ■ Is database slow?

- Check RDS CloudWatch metrics: CPUUtilization, ReadLatency, WriteLatency, DatabaseConnections.
- High connection count could indicate connection pool exhaustion — too many app pods connecting.
- Slow queries cause the application to wait — check RDS Performance Insights for long-running queries.

#### ■ Is network latency present?

- Check ALB TargetResponseTime metric in CloudWatch — measures latency between ALB and your pods.

- Check inter-service latency if microservices architecture — one slow downstream service affects all callers.
- Check VPC Flow Logs for retransmits or packet drops.

#### ■ Which metrics would you investigate?

- Application: p99 response time (99th percentile latency), error rate (5xx responses).
- Pod: CPU utilization vs limits, memory utilization vs limits.
- Database: RDS read/write latency, connection count, CPU utilization.
- Load Balancer: ALB TargetResponseTime, HTTPCode\_ELB\_5XX\_Count.
- Network: bytes in/out, packet retransmit rate.

### Scenario 17: Prometheus Stops Collecting Metrics

#### ■ What components would you check?

- First, check if Prometheus itself is running:  
`kubectl get pods -n monitoring`
- Check Prometheus pod logs for errors:  
`kubectl logs <prometheus-pod> -n monitoring --tail=50`
- Check if ServiceMonitor or PodMonitor CRDs are still present.
- Check if the Prometheus config was recently changed (wrong scrape config).

#### ■ How does service discovery work?

- Prometheus uses ServiceMonitor and PodMonitor CRDs (Custom Resource Definitions) to discover targets.
- It selects pods/services that match label selectors defined in the ServiceMonitor.
- Alternatively, pods can use annotations: 'prometheus.io/scrape: true' with the right port.
- Prometheus periodically queries the Kubernetes API to get the current list of targets.

#### ■ How would you verify targets?

- Port-forward to the Prometheus UI and check Status → Targets:  
`kubectl port-forward svc/prometheus-operated 9090:9090 -n monitoring`
- Open `http://localhost:9090/targets` in your browser.
- Targets showing as 'Down' with an error message tell you exactly what's failing.
- Targets showing as 'Unknown' mean Prometheus can't reach them at all.

#### ■ How would you troubleshoot exporters?

- Check if the exporter pod is running (e.g., node-exporter, kube-state-metrics):  
`kubectl get pods -n monitoring`
- Manually test the metrics endpoint from inside the cluster:  
`kubectl exec -it <any-pod> -- curl http://<exporter-ip>:9100/metrics`
- Verify the port and path in the ServiceMonitor match the actual port the exporter exposes.
- Check exporter logs for scrape errors or panics.



## SECTION 8 — GitHub Actions and GitOps Scenarios

### Scenario 18: Argo CD Shows OutOfSync

#### ■ Why might this happen?

- Someone ran 'kubectl apply' directly on the cluster, bypassing Git — this is the most common cause.
- A Helm chart was manually upgraded or values were edited directly.
- A Kubernetes object was manually scaled (kubectl scale) without a Git commit.
- The application generates runtime state that Argo CD detects as a diff (e.g., generated labels).

#### ■ How would you investigate?

- Open the Argo CD UI → click on the application.
- The 'Diff' button shows a side-by-side comparison: Git state (desired) vs cluster state (live).
- This tells you exactly which field was changed and what the difference is.
- Check recent Git commits — was there a recent change that should have triggered a sync but didn't?

#### ■ What logs would you check?

- Argo CD application controller logs — shows sync errors and decisions:  

```
kubectl logs -n argocd -l app.kubernetes.io/name=argocd-application-controller --tail=50
```
- Argo CD repo server logs (if sync is failing due to a Git issue):  

```
kubectl logs -n argocd -l app.kubernetes.io/name=argocd-repo-server --tail=50
```
- The Argo CD UI also shows events and conditions per application resource.

#### ■ How would you restore synchronization?

- If Git is the source of truth (as it should be in GitOps): click 'Sync' in Argo CD UI — this reverts manual changes and reapplies Git state.
- If the manual change was intentional: commit that change to Git first, then sync — this keeps Git accurate.
- Enable auto-sync in Argo CD to prevent future drift: Argo CD will automatically reapply Git state whenever it detects a diff.

### Scenario 19: A Bad Commit Was Deployed to Production

#### ■ How would GitOps help rollback?

- In GitOps, every deployment is a Git commit — the full deployment history is the Git history.
- You can identify the last good commit hash and revert to it.
- Argo CD immediately syncs after the revert commit is pushed — deployment happens automatically.
- Full audit trail: you know exactly what was deployed, when, and by whom.

#### ■ Would you revert the commit?

- Yes — 'git revert' is the preferred approach in GitOps. It creates a new commit that undoes the bad commit.  

```
git revert <bad-commit-hash>
git push origin main
```
- This is safer than 'git reset' because it doesn't rewrite history — the CI/CD pipeline re-runs cleanly.
- Argo CD detects the new commit and syncs the reverted state to the cluster.

#### ■ Would you rollback the deployment?

- A quick kubectl rollout undo is possible for immediate mitigation while the Git fix is being prepared.  

```
kubectl rollout undo deployment/<name> -n <namespace>
```
- However, this creates an OutOfSync state in Argo CD — Git and cluster are now different.
- This is acceptable as a temporary emergency action, but must be followed by a Git revert.

#### ■ What is the safest option?

- Two-step approach: (1) kubectl rollout undo for immediate recovery, (2) git revert + push to fix Git.

- Single-step if time allows: git revert + push → let Argo CD handle everything — cleanest approach.
  - Never force-push or git reset on main in production without team awareness.
-

## SECTION 9 — Microservices Scenarios

### Scenario 20: Order Service is Down, Frontend is Still Available

#### ■ How would you identify the issue?

- Check Order Service pod status immediately:  
`kubectl get pods -n <namespace> | grep order`
- If pods are CrashLoopBackOff or Error: read logs to find the root cause.
- If pods are Running but service is unavailable: check if the Service endpoints are populated.  
`kubectl get endpoints order-service -n <namespace>`

#### ■ What logs would you check?

- Order Service pod logs — look for database connection errors, startup exceptions, timeout errors.  
`kubectl logs <order-pod-name> -n <namespace>`
- Previous pod logs if the pod keeps restarting:  
`kubectl logs <order-pod-name> --previous -n <namespace>`
- Kubernetes events for the namespace:  
`kubectl get events -n <namespace> --sort-by='.lastTimestamp'`

#### ■ Could it be a database issue?

- Yes — look for 'connection refused' or 'connection timeout' in the Order Service logs.
- Check RDS status in AWS Console — is it available?
- Check Security Groups: EKS nodes need outbound access to RDS on port 5432 (PostgreSQL) or 3306 (MySQL).
- Check if DB credentials in the Kubernetes Secret are still correct (they may have been rotated).

#### ■ Could it be a networking issue?

- Check if DNS resolves the Order Service from within another pod:  
`kubectl exec -it <frontend-pod> -- nslookup order-service.<namespace>.svc.cluster.local`
- Check NetworkPolicies — a recently added NetworkPolicy may be blocking ingress/egress.
- Check if the Service selector matches the pod labels.

#### ■ How would you restore service quickly?

- If a bad deployment caused it: rollback immediately.  
`kubectl rollout undo deployment/order-service -n <namespace>`
- If it's a config issue: fix the ConfigMap/Secret and restart the pods.  
`kubectl rollout restart deployment/order-service -n <namespace>`
- If it's a DB issue: fix the connectivity or credentials, then restart pods.
- Communicate the incident to the team — don't work in silence during a production outage.

### Scenario 21: Two Microservices Cannot Communicate

#### ■ How would you troubleshoot?

- Verify both pods are Running and Ready (not just Running):  
`kubectl get pods -n <namespace>`
- Test connectivity directly from one pod to the other:  
`kubectl exec -it <pod-a> -- curl http://<service-b>:<port>/health`
- Check DNS resolution from within a pod:  
`kubectl exec -it <pod-a> -- nslookup <service-b>.<namespace>.svc.cluster.local`

#### ■ What Kubernetes resources would you inspect?

- Service for service-b: does it exist? Is the port correct?  
`kubectl describe svc <service-b> -n <namespace>`

- Endpoints for service-b: are they populated? Empty endpoints = selector mismatch.

```
kubectl get endpoints <service-b> -n <namespace>
```

- NetworkPolicy: any policy that might be blocking ingress to service-b?

```
kubectl get networkpolicy -n <namespace>
```

### ■ How would DNS affect communication?

- Kubernetes has built-in DNS via CoreDNS. Services are accessible as:

<service>.<namespace>.svc.cluster.local

- Using just the service name (e.g., 'order-service') only works within the same namespace.
- Cross-namespace communication requires the full DNS name: order-service.orders.svc.cluster.local
- If CoreDNS is crashing, all DNS lookups in the cluster fail:

```
kubectl get pods -n kube-system | grep coredns
```

### ■ How would you test connectivity?

- Use curl from inside a pod:

```
kubectl exec -it <pod-a> -- curl -v http://<service-b>:<port>/
```

- Deploy a temporary debugging pod if the target pods don't have curl:

```
kubectl run debug --image=curlimages/curl -it --rm -- sh
```

- Test with wget as alternative:

```
kubectl exec -it <pod-a> -- wget -qO- http://<service-b>:<port>/health
```

---

## SECTION 10 — CI/CD Pipeline Design Scenarios

### Scenario 22: Design CI/CD from Scratch (No Tools, No Kubernetes)

#### ■ How would you design a CI/CD pipeline?

- The pipeline has two parts: CI (Continuous Integration) and CD (Continuous Delivery).
- CI: triggered on every code push — build the application, run tests, scan for vulnerabilities, build Docker image.
- CD: triggered after CI passes — deploy the image to the target environment.
- Use GitOps: store Kubernetes manifests in a separate Git repo. Argo CD watches that repo and deploys automatically.

#### ■ What tools would you introduce?

- GitHub / GitHub Actions — source control and CI pipeline (free tier available, no extra infra needed).
- Docker — containerize the application.
- AWS ECR — private Docker image registry (secure, integrated with EKS and IAM).
- Terraform — provision EKS, VPC, RDS, and all AWS infrastructure as code.
- Amazon EKS — managed Kubernetes for running containerized workloads.
- Argo CD — GitOps-based continuous delivery to Kubernetes.
- Prometheus + Grafana — monitoring and alerting.
- SonarQube + Trivy — code quality and container security scanning.

#### ■ Why these tools?

- GitHub Actions is built into GitHub — no extra server needed, unlike Jenkins which requires its own maintenance.
- Terraform makes infrastructure repeatable and version-controlled — you can recreate the whole infra from code.
- Argo CD ensures Git is the single source of truth — every deployment is auditable.
- EKS reduces Kubernetes operational overhead — AWS manages the control plane.

#### ■ What would your deployment workflow look like?

- Step 1: Developer pushes code to GitHub → GitHub Actions workflow triggers.
- Step 2: Run unit tests → SonarQube code quality scan → Trivy image security scan.
- Step 3: Build Docker image and push to AWS ECR with commit SHA as tag.
- Step 4: Update image tag in Kubernetes manifest Git repository (automated via GitHub Actions).
- Step 5: Argo CD detects the new image tag in Git → syncs to EKS → new pods roll out.
- Step 6: Prometheus and Grafana monitor the deployment health.

### Scenario 23: Multi-Environment Pipeline (Dev, QA, Staging, Production)

#### ■ How would you structure Git branches?

- feature/\* → developer creates feature branches for all new work.
- develop → feature branches are merged here via PR. Auto-deploys to Dev environment.
- qa → merges from develop after Dev testing passes. Auto-deploys to QA environment.
- staging → merges from qa after QA sign-off. Auto-deploys to Staging.
- main → merges from staging only. Deploys to Production with manual approval.

#### ■ How would you structure Jenkins pipelines?

- One Jenkinsfile with conditional stages based on the branch name.
- Dev: auto-trigger on push to develop — build, test, deploy to dev namespace.
- QA: auto-trigger after Dev pipeline passes — deploy to qa namespace.
- Staging: auto-trigger after QA pipeline passes — deploy to staging namespace.

- Production: trigger manually with an approval gate — 'input' step in Jenkinsfile requires human confirmation.

```
input message: 'Deploy to Production?', ok: 'Approve'
```

#### ■ How would you manage Terraform environments?

- Separate directories for each environment: terraform/dev/, terraform/qa/, terraform/staging/, terraform/prod/
- Shared modules are in terraform/modules/ — used by all environments to avoid code duplication.
- Each environment has its own terraform.tfvars with environment-specific values (instance size, replica count).
- Each environment has its own S3 state file to prevent cross-environment state corruption.

#### ■ How would you handle approvals?

- Jenkins: 'input' step before production deployment — requires a named approver to click 'Approve'.
  - GitHub: branch protection rules on 'main' — require 1+ reviewers before merge is allowed.
  - Argo CD: set sync policy to 'manual' for the production application — no auto-sync.
  - Slack notifications: send a message to #deployments when a production deploy is pending approval.
-

## SECTION 11 — Leadership and Behavioral Scenarios

### Scenario 24: Developer Wants to Bypass Code Review Due to Urgency

#### ■ What would you do?

- Have a direct, calm conversation: acknowledge the urgency, but explain the purpose of code review.
- Code review catches bugs, security issues, and logic errors that the developer themselves may have missed — especially under pressure.
- Propose an alternative: an expedited review where one senior engineer reviews in 15 minutes.

#### ■ Would you allow direct merge?

- No, not as a standard exception. Urgency is exactly when mistakes are most likely.
- The correct path is an emergency/hotfix process — pre-defined in team policy — that streamlines review without eliminating it.
- If branch protection rules are configured correctly, even admins need to override consciously (creating an audit trail).

#### ■ How would you balance speed and safety?

- Define an emergency merge process in advance (before an incident): one reviewer, 15-minute SLA, post-merge review.
- Make branch protection rules enforce at least one reviewer — this cannot be bypassed accidentally.
- After the urgent fix, do a proper post-merge review to catch anything that was missed.

*Note: Showing you care about process and safety — even under pressure — is a strong signal for any DevOps role.*

### Scenario 25: Production Issue at Midnight

#### ■ How do you respond?

- Acknowledge the alert immediately — even just 'I see the alert, investigating' in the incident channel.
- Assess severity: how many users are affected? Is the entire service down or a partial outage?
- Start systematic troubleshooting: check monitoring dashboards, pod status, recent deployments.
- Don't make random changes hoping something fixes it — diagnose before acting.

#### ■ How do you communicate updates?

- Post clear status updates every 15-30 minutes in the incident Slack channel — silence is worse than bad news.
- Format: 'Status: [Investigating / Identified / Fixing / Resolved]. Impact: [X users]. Next update in 15 min.'
- Separate the technical investigation from stakeholder communication — one person investigates, one communicates.

#### ■ Who do you involve?

- Immediately: the on-call DevOps or SRE engineer.
- If database issue: on-call DBA.
- If impact is severe (full outage, data loss risk): wake up team lead and management — don't manage this alone.
- If security incident: notify the security team immediately.
- Rule: when in doubt, over-communicate. It's always better to loop someone in and have them say 'I'm not needed' than to not loop them in.

#### ■ How do you document the incident?

- During the incident: keep a running timeline of every action taken with timestamps.
- After resolution: write a Post-Incident Report (PIR) / Post-Mortem document.
- PIR must include: timeline, root cause, impact (users affected, duration), resolution steps, and — most importantly — prevention actions.

- Share the PIR with the team within 48 hours. The goal is blameless learning, not blame.

*Note: Being able to describe a good incident response process, even without direct experience, shows maturity and readiness.*

---

## Scenario 26: Two Engineers Disagree on Deployment Strategy

### ■ How would you handle the disagreement?

- Listen to both sides fully without interrupting — understand the technical reasoning behind each position.
- Ask both engineers to explain the trade-offs of their approach: reliability, complexity, cost, rollback capability.
- Keep the discussion focused on technical merit and team goals — not personal preferences.

### ■ How would you reach a decision?

- Document both approaches with clear pros and cons — this structures the conversation.
- If the team can't agree: escalate to the tech lead or senior engineer for a final decision.
- If time allows: run a spike (time-boxed technical exploration) to validate the key assumption.
- Accept the decision once it's made — commit to making it work, even if it wasn't your preferred approach.

### ■ What factors would influence your choice?

- Reliability and rollback: how quickly can we recover if this deployment strategy fails?
  - Team expertise: does the team have the skills to maintain this approach?
  - Maintenance overhead: does this add long-term complexity to operations?
  - Cost: does one approach cost significantly more?
  - Timeline: is there a deadline that affects the decision?
  - Compliance/security requirements: does either approach create a compliance risk?
-



## SECTION 12 — Deep Cross-Questions Interviewers Ask

These follow-up questions are asked immediately after you mention a keyword. Prepare these so you don't get caught by your own answers.

### If You Mention HPA (Horizontal Pod Autoscaler)

#### ■ What metrics does HPA use?

- CPU utilization — the most common metric, expressed as a percentage of requested CPU.
- Memory utilization — available from Kubernetes 1.23+ natively.
- Custom metrics — via Prometheus Adapter: queue length, requests per second, active connections, or any Prometheus metric.

#### ■ How frequently does HPA evaluate metrics?

- Every 15 seconds by default — controlled by the '--horizontal-pod-autoscaler-sync-period' flag on the controller manager.
- Scale-up happens quickly, but scale-down has a stabilization window of 5 minutes by default to avoid flapping.

#### ■ What are HPA's limitations?

- Cannot scale to zero pods — minimum is 1. Use KEDA (Kubernetes Event-Driven Autoscaler) if you need scale-to-zero.
- Scale-down is intentionally slow — the 5-minute window prevents rapid up-down oscillation.
- Works poorly with stateful applications that can't handle sudden replica changes.
- Requires the Kubernetes Metrics Server to be installed and running.

### If You Mention Amazon EKS

#### ■ How does EKS differ from self-managed Kubernetes?

- EKS: AWS manages the control plane (API server, etcd, scheduler, controller manager) — highly available across multiple AZs.
- Self-managed: you provision and maintain the control plane yourself on EC2 — much more operational overhead.
- EKS: deep integration with AWS services (IAM, ALB, EBS, VPC, CloudWatch).
- Self-managed: more flexibility but more work — you're responsible for control plane upgrades, backups, and HA.

#### ■ What components does AWS manage in EKS?

- API server, etcd (distributed state store), kube-scheduler, kube-controller-manager.
- These run on AWS-managed infrastructure, automatically distributed across 3 AZs.
- You manage: worker nodes, add-ons (CoreDNS, kube-proxy, CNI), and your workloads.

#### ■ How does authentication work in EKS?

- Authentication uses AWS IAM: kubectl authenticates by getting a short-lived token via the AWS CLI.
- Authorization uses Kubernetes RBAC: the aws-auth ConfigMap maps IAM users/roles to Kubernetes users/groups.

```
kubectl edit configmap aws-auth -n kube-system
```

- For pod-level AWS access: IRSA (IAM Roles for Service Accounts) — pods get AWS permissions via annotated service accounts.

### If You Mention Terraform

#### ■ How is Terraform state stored?

- Default: locally in 'terraform.tfstate' — fine for solo projects, dangerous for teams.
- Recommended for teams: remote backend using S3 (state storage) + DynamoDB table (state locking).  

```
terraform { backend "s3" { bucket = "tf-state" key = "prod/terraform.tfstate" region = "ap-south-1" dynamodb_table = "tf-lock" } }
```

### ■ What happens if state is lost?

- Terraform loses track of existing infrastructure — next 'terraform apply' tries to create everything again, causing duplicates.
- Recover using 'terraform import' to re-add existing resources to a new state file.
- This is why S3 versioning is essential — you can restore any previous version of the state file.

### ■ How do modules work internally?

- A module is a folder of .tf files. You call it using a 'module' block and pass variables as arguments.
- Modules have input variables (var.x), output values (output.y), and their own resource definitions.
- The parent module can use the child module's outputs: module.vpc.vpc\_id
- All module resources are stored in the same state file as the root module — there's no separate state per module.

## If You Mention Jenkins

### ■ Where are credentials stored in Jenkins?

- Jenkins Credentials Store — accessible via Manage Jenkins → Credentials.
- Encrypted at rest using the Jenkins secret key.
- Referenced in Jenkinsfile using: credentials('my-secret-id') — the actual value is never exposed in logs.
- Can also integrate with HashiCorp Vault or AWS Secrets Manager for more robust secret management.

### ■ How are Jenkins agents provisioned?

- Static agents: configured permanently in Manage Jenkins → Manage Nodes — always online.
- Dynamic agents using Kubernetes plugin: a new pod is created for each build, deleted after completion.
- Dynamic agents using Docker plugin: a new container is spun up per build.
- Dynamic agents are preferred — they scale automatically and don't waste resources when idle.

### ■ How do you secure Jenkins?

- Enable authentication — use LDAP, SSO (Google, GitHub), or local accounts. Disable anonymous access.
- Install the Role-Based Access Control (RBAC) plugin — restrict who can configure jobs, manage nodes, etc.
- Use HTTPS for the Jenkins URL — never serve Jenkins over plain HTTP.
- Keep Jenkins core and plugins updated — plugins are a major source of vulnerabilities.
- Store all secrets in Credentials Store — never hardcode credentials in Jenkinsfiles.

## If You Mention Docker

### ■ What happens during 'docker build'?

- Docker reads the Dockerfile from top to bottom, executing each instruction in order.
- Each instruction (RUN, COPY, ADD) creates a new read-only layer on top of the previous one.
- Before executing each instruction, Docker checks its layer cache — if the instruction and inputs match a cached layer, it uses the cache (much faster).
- The final image is a union of all layers.

### ■ What layers are created?

- FROM: sets the base layer (e.g., python:3.11-slim).
- RUN: each RUN instruction creates one layer containing the filesystem changes from that command.
- COPY / ADD: copies files into the image, creating a layer with those files.
- ENV, LABEL, EXPOSE: metadata instructions — they create very thin layers with no filesystem change.

### ■ How does Docker layer caching work?

- Docker hashes each instruction + its inputs. If the hash matches a cached layer, the cache is used.
- Cache invalidation is sequential: if one layer's cache is invalidated (e.g., COPY of changed files), all subsequent layers are also rebuilt.
- Best practice: put COPY of frequently-changing files (source code) as late as possible in the Dockerfile to maximize cache hits for dependency installation.

---

## If You Mention AWS IAM

### ■ How does IAM policy evaluation work?

- Default: everything is denied. No permission is granted unless explicitly allowed.
- Evaluation order: Service Control Policies (SCPs) → Resource-based policies → Identity-based policies → Session policies.
- Explicit Allow + no Deny = access granted.
- Explicit Deny anywhere in the chain = access denied, regardless of any allow.

### ■ What is an explicit deny?

- A policy statement with 'Effect: Deny' — this unconditionally blocks the action.
- It cannot be overridden by any allow statement, even from another policy or even AdministratorAccess.
- Used in SCPs to enforce organizational guardrails: e.g., 'No one can delete CloudTrail logs', even the root account (via SCP).

### ■ What is a permission boundary?

- A permission boundary sets the maximum permissions an IAM entity (user or role) can ever have.
  - Even if the entity's policies grant broader access, the boundary limits the effective permissions to the intersection.
  - Use case: you delegate user creation to a junior admin but use permission boundaries to ensure they can't create users with more permissions than they have.
-

## QUICK REFERENCE — Most Important Commands by Topic

| Topic      | Situation             | Command / Action                                                |
|------------|-----------------------|-----------------------------------------------------------------|
| Kubernetes | Pod crashing          | kubectl logs <pod> --previous                                   |
| Kubernetes | Pod not starting      | kubectl describe pod <pod>;                                     |
| Kubernetes | Service not working   | kubectl get endpoints <svc>;                                    |
| Kubernetes | Rollback deployment   | kubectl rollout undo deployment/<name>;                         |
| Kubernetes | Drain a node          | kubectl drain <node>; --ignore-daemonsets                       |
| Kubernetes | Check resource usage  | kubectl top pods / kubectl top nodes                            |
| EKS        | Node is NotReady      | kubectl describe node + check EC2 in AWS Console                |
| EKS        | ALB not created       | kubectl logs -n kube-system -l app=aws-load-balancer-controller |
| Terraform  | Check drift           | terraform plan                                                  |
| Terraform  | Sync state with infra | terraform refresh                                               |
| Terraform  | Fix state entries     | terraform state rm / terraform import                           |
| IAM        | Find access denied    | AWS CloudTrail filter by AccessDenied                           |
| IAM        | Test permissions      | AWS IAM Policy Simulator                                        |
| Docker     | Find large layers     | docker history <image>;                                         |
| Argo CD    | Check sync diff       | Argo CD UI → Diff view                                          |
| Argo CD    | Force sync            | argocd app sync <app-name>;                                     |
| Prometheus | Check targets         | Prometheus UI → Status → Targets                                |
| Jenkins    | Debug pipeline        | Jenkins UI → Console Output for failed build                    |

*This guide covers all 26 scenario-based interview questions tailored to your internship profile at Hisan Labs. Focus on Kubernetes troubleshooting (Section 2), Terraform state (Section 4), and GitOps/Argo CD (Section 8) — these are your strongest areas and interviewers will go deepest there.*